

NATIONAL RESEARCH UNIVERSITY HIGHER SCHOOL OF ECONOMICS

SCHOOL OF COMPUTER SCIENCE, PHYSICS AND TECHNOLOGIES

FACULTY OF APPLIED MATHEMATICS AND COMPUTER SCIENCE

PROJECT PROPOSAL

VLA KNOWLEDGE TRANSFER WITH TRAJECTORY GUIDANCE

Student:

Saprygin Igor, BAM222C

Supervisor:

Daniil Paleev

SAINT-PETERSBURG

2026

Abstract

Vision–Language–Action (VLA) models have recently emerged as a promising approach for training robots to perform tasks specified through natural language. However, these models typically require large amounts of training data and substantial computational resources. In this work, we consider the following setting: given a robot *A*, a VLA policy trained to manipulate it and execute natural language instructions (e.g., object picking or sorting), and a simulator providing access to both the robot and the policy, we aim to leverage this setup to train a policy for a different robot *B* with a distinct embodiment. We propose to use robot *A* as a generator of gripper trajectories, which are largely embodiment-agnostic and can therefore serve as a component of the reward signal for training policies controlling other robots. These trajectories can be incorporated into standard reinforcement learning algorithms to guide policy learning for robot *B*. Consequently, this work focuses on reinforcement learning–based transfer and compares the proposed approach with alternative methods for cross-embodiment policy transfer.

1 Introduction

Recent advances in robot learning have been driven by the emergence of Vision–Language–Action (VLA) models, which aim to connect visual perception, natural language understanding, and robot control within a unified framework. These models enable robots to interpret high-level language instructions and translate them into sequences of actions that achieve the desired task. As a result, VLAs have become a promising paradigm for building more flexible and general-purpose robotic systems capable of performing a wide variety of manipulation tasks.

Despite these capabilities, training VLA models typically requires large amounts of interaction data and substantial computational resources. Collecting such data on real robots is expensive and time-consuming, while training large models in simulation also incurs significant computational cost. Moreover, policies trained for a specific robot embodiment often do not generalize well to other robots with different kinematic structures, actuator limits, or sensor configurations. As a result, transferring policies between robots remains a challenging problem in robot learning.

VLAs are usually trained by cloning expert actions given the observation. In this work, we instead focus on reinforcement learning approaches. However, pure reinforcement learning is difficult because the policy initially behaves almost randomly and therefore rarely receives meaningful reward signals. One possible approach to this problem is to leverage existing trained policies as sources of structured behavioral data. In particular, trajectories generated by a robot performing a task can contain high-level information about how the task should be accomplished. If such information can be extracted in a form that is independent of a specific embodiment, it may be possible to reuse it when training policies for different robotic platforms.

In this work, we consider a scenario where a robot *A* and a VLA policy trained to control it are available in simulation. Our goal is to leverage this policy to accelerate learning for a different robot *B* with a different embodiment. Instead of directly transferring the policy, we propose to use robot *A* as a generator of task-relevant gripper trajectories. These trajectories represent the motion of the end-effector and are therefore largely independent of the specific robot configuration. We hypothesize that such trajectories can serve as useful guidance for training policies for other robots.

To incorporate this information into learning, we propose to use the generated trajectories as part of the reward signal in a reinforcement learning framework. By encouraging robot B to reproduce task-relevant end-effector motion patterns, reinforcement learning algorithms may learn effective manipulation strategies even when the underlying robot embodiment differs from that of the source robot.

The main objective of this project is therefore to investigate whether trajectory guidance can improve cross-embodiment policy transfer. In particular, we will incorporate trajectory-based rewards into a reinforcement learning algorithm and compare the proposed approach with alternative transfer strategies.

2 Literature Review

2.1 Vision-Language-Action Models in Robotics

The idea of Vision–Language–Action (VLA) models can be seen as a natural extension of the recent progress in large-scale language and multimodal models. The development of large language models (LLMs), enabled in part by the Transformer architecture introduced in “Attention Is All You Need” [1], demonstrated that a single model can learn rich representations of text from large datasets. Subsequent work extended these ideas to multimodal settings, giving rise to Vision–Language Models (VLMs) that jointly process images and text. One of the most influential examples is CLIP [2], which showed that aligning visual and textual representations in a shared embedding space allows models to capture semantic relationships between images and natural language.

VLA models extend this paradigm by adding an additional modality: actions. In these models, the output space is no longer text but a sequence of robot control actions. Many VLA systems, such as RT-2 [3] and OpenVLA [4], build on pretrained VLM architectures and adapt them to produce actions in a robot’s control space. More lightweight architectures, such as SmolVLA [5], aim to reduce computational requirements while preserving the ability to map multimodal observations to control policies, making them more practical for research and deployment on smaller robotic platforms.

The resulting policy interacts with the environment through a closed-loop process in which the robot observes the current state (e.g., camera images and joint states), produces an action, executes it, and receives new observations. This interaction loop can be written as

$$o_t \rightarrow a_t \rightarrow o_{t+1},$$

where o_t denotes the observation at time step t and a_t the corresponding action.

Such VLA models have demonstrated the ability to perform complex manipulation tasks specified in natural language, including object sorting, folding clothes, and other multi-step robotic behaviors.

2.2 Reinforcement Learning

Reinforcement learning (RL) has become a widely used framework for learning control policies in robotics. In this setting, we consider an environment (which can be a game, a simulator, or a real robotic system) and an agent (e.g., a robot). At every step, the agent observes the current state of the environment (for example, a camera frame or the robot’s joint configuration) and selects an action according to its policy, which is typically represented by a neural network. The action may correspond to sending control signals to the robot’s joints or operating the gripper. The environment then returns a reward signal indicating how desirable the chosen action was in the current state. This interaction forms the so-called RL interaction loop, in which the agent gradually learns to select actions that maximize cumulative reward.

Formally, reinforcement learning problems are often modeled as a Markov Decision Process (MDP), defined by the tuple

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma),$$

where $\mathcal{S}$ denotes the set of states, $\mathcal{A}$ the set of actions, $P(s' \mid s, a)$ the transition dynamics, $r(s, a)$ the reward function, and $\gamma \in [0, 1]$ the discount factor. The agent’s behavior is described by a policy $\pi(a \mid s)$, which specifies a distribution over actions given a state.

RL algorithms such as PPO [6] and SAC [7] have been successfully applied to a variety of robotic manipulation tasks, including grasping, object rearrangement, and locomotion.

Generally speaking, the objective of reinforcement learning is to learn a policy π that maximizes the expected cumulative reward over trajectories τ generated by the policy:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

Here, r_t denotes the reward at time step t , and $\gamma \in [0, 1]$ is the discount factor.

Despite these advances, applying reinforcement learning to robotic manipulation remains challenging due to sparse reward signals and high sample complexity. In many tasks, the most natural reward formulation is sparse, for example

$$r(s_t, a_t) = \begin{cases} 1 & \text{if the task is successfully completed,} \\ 0 & \text{otherwise,} \end{cases}$$

which is difficult to optimize because the agent rarely encounters successful states during exploration. As a result, many recent approaches attempt to incorporate additional guidance into the learning process, such as demonstrations, trajectory priors, or structured reward signals. One such approach is HIL-SERL [8], which aims to reduce data requirements by incorporating human feedback during policy rollouts, allowing an operator to correct the robot’s behavior as it learns to complete the task.

In this work, we explore a related idea by leveraging trajectories generated by an existing robot policy to guide reinforcement learning for a different robot embodiment. In some sense, we remove the human operator from the loop and instead utilize knowledge encoded in a trained robot policy.

2.3 Knowledge Transfer Between Embodiments

A major challenge in robot learning is transferring knowledge between robots with different embodiments. Policies trained for a particular robot often depend on its specific kinematic structure, action space, and sensor configuration, which makes direct transfer to another robot difficult. Formally, a policy $\pi(a \mid s)$ trained for one embodiment assumes a particular state space $\mathcal{S}$ and action space $\mathcal{A}$, and these spaces may differ significantly between robots.

One line of research aims to learn shared representations or generalist policies that can generalize across robots with different morphologies [9]. Another approach relies on learning embodiment-specific corrections that are added to a base policy

$$\pi_B(a \mid s) = \pi_{\text{base}}(a \mid s) + \Delta_{\text{emb}}(a \mid s),$$

where Δ_{emb} represents an embodiment-specific adjustment [10]. Recent work also explores embodiment-agnostic policy learning, with the goal of capturing task-relevant structure that can be reused across different robotic platforms [11].

However, direct policy transfer remains challenging when the source and target robots differ substantially in their physical configuration, since their low-level control spaces may be incompatible and it still requires some amount of manually collected data. This motivates the exploration of alternative forms of knowledge transfer based on higher-level task signals rather than direct reuse of policy parameters.

2.4 Dense Reward Shaping

As mentioned earlier, sparse rewards are difficult to optimize because they provide little feedback and do not measure intermediate progress. Several approaches attempt to construct dense reward functions that provide a smoother learning signal. For example, reward classifiers used in approaches such as HIL-SERL estimate the probability that an observation belongs to a successful trajectory,

$$c(s) = P(\text{success} \mid s),$$

which can then be converted into a dense reward signal, for example

$$r(s) = \log c(s).$$

Similarly, methods such as GAIL [12] learn reward signals from demonstrations by training a discriminator that distinguishes expert trajectories from those generated by the current policy. Other works, such as DeepMimic [13], demonstrate that dense rewards based on similarity to expert trajectories can significantly improve learning. In such settings, the reward can be defined as the distance between the agent’s state and an expert trajectory,

$$r_t = -\|x_t - x_t^{\text{expert}}\|^2,$$

where x_t represents the agent’s state or end-effector position at time step t .

However, a limitation of these approaches is that expert trajectories often need to be recorded by a human operator. Our approach can be viewed as an extension of this idea, where the expert trajectories are generated automatically by a trained robot policy rather than collected through human demonstrations.

3 Methods

3.1 Problem Formulation

We consider the problem of transferring manipulation knowledge between two robots with different embodiments. Let robot A denote a source robot for which a trained policy $\pi_A(a | s)$ is available, and robot B denote a target robot for which we aim to learn a new policy $\pi_B(a | s)$. The goal is to leverage knowledge encoded in π_A to accelerate the learning process for robot B .

The interaction between the robot and the environment is modeled as an MDP.

We train the target policy using the standard reinforcement learning objective introduced earlier. The key difference in our formulation lies in the reward function, which incorporates trajectory information generated by the source robot.

3.2 Simulation Environment



Figure 1: Isaac Sim environment with SO-101 arms training in parallel

All experiments are conducted in simulation using NVIDIA Isaac Sim, a physics-based robotics simulator designed for training and evaluating robotic policies. Isaac Sim provides high-fidelity physics simulation, realistic rendering, and integration with modern machine learning frameworks, making it suitable for reinforcement learning research.

The simulated environment contains the robot, manipulable objects, and sensors such as RGB cameras. The simulator provides observations that include visual inputs from the cameras as well as robot state information such as joint positions and velocities. Given an action issued by the policy, the simulator advances the system according to its underlying physics model and produces the next observation.

Using simulation allows us to generate large numbers of interaction trajectories efficiently and safely before transferring the learned policies to real robotic platforms. The simulator also enables controlled experimentation when comparing different reward formulations and training procedures.

3.3 Robotic Platforms



(a) Source: SO101



(b) Target: Piper

Figure 2: Robotic embodiments considered in this work.

Our experiments involve two robotic embodiments: a source robot and a target robot. The source robot is the SO-101 robotic arm, which is used to generate expert trajectories for the manipulation task. The target robot is the AgileX Piper arm, on which the reinforcement learning policy is trained.

These robots differ in their kinematic structure, joint configuration, and control interfaces. As a result, policies trained directly on one robot cannot be transferred to the other without adaptation. This embodiment mismatch motivates the use of trajectory-based guidance rather than direct policy transfer.

The SO-101 robot is used to collect successful task trajectories that serve as a source of supervisory signal. These trajectories provide information about task progress and desired behaviors without requiring human demonstrations.

The AgileX Piper robot operates in a copy of the same simulated task environment and uses its own control space and dynamics. During training, the reinforcement learning agent controlling

the Piper arm receives rewards derived from the similarity between its trajectories and the expert trajectories generated by the SO-101 robot.

3.4 Policy Architecture

In our experiments, we use SmolVLA [5] as the base Vision–Language–Action model. SmolVLA is a compact VLA architecture designed to retain the core capabilities of larger VLA systems while remaining computationally efficient. This makes it particularly suitable for research environments with limited computational resources. In our case, the model can be trained using the GPU resources available in our laboratory while still producing meaningful behavioral results in manipulation tasks.

The model receives multimodal observations including camera images, robot state information, and task instructions, and produces actions in the robot’s control space.

The policy maps observations o_t to control actions a_t that correspond to commands for the robot’s actuators. During training, the policy is optimized using reinforcement learning with the trajectory-based reward described in the previous subsection.

3.5 Trajectory Extraction from the Source Policy

To obtain guidance for learning, we first generate trajectories using the trained source policy π_A . Executing π_A in the environment produces trajectories of the form

$$\tau^A = \{(s_0^A, a_0^A), (s_1^A, a_1^A), \dots, (s_T^A, a_T^A)\}.$$

From these trajectories we extract task-relevant features such as the end-effector pose or other task-space representations. These trajectories serve as demonstrations of successful task execution.

3.6 Trajectory-Based Reward

To guide learning for the target robot, we incorporate a trajectory-based reward that encourages the policy to follow motion patterns produced by the source robot. However, instead of replacing the task reward entirely, we combine this signal with the original sparse task reward provided by the environment.

Let x_t^A denote the task-space state (e.g., end-effector pose) of robot A at time step t , and x_t^B the corresponding state of robot B . We define a trajectory similarity reward as

$$r_t^{\text{traj}} = -\|x_t^B - x_t^A\|^2.$$

This term encourages the target robot to follow a trajectory similar to that produced by the source policy while allowing adaptation to the target embodiment.

The final reward is defined as a weighted combination of the sparse task reward r_t^{task} and the trajectory-based reward:

$$r_t = r_t^{\text{task}} + \lambda r_t^{\text{traj}},$$

where λ controls the strength of the trajectory guidance. The value of λ will be selected empirically during training.

3.7 Training Procedure

The target policy π_B is trained using a reinforcement learning algorithm such as Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC). During training, the agent interacts with the environment and receives the trajectory-based reward defined above.

The learning objective is therefore

$$\max_{\pi_B} \mathbb{E}_{\tau \sim \pi_B} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

where r_t incorporates the trajectory similarity signal.

3.8 Evaluation Setup

To evaluate the proposed approach, we consider several robotic manipulation tasks such as object picking, sorting, and other language-conditioned tasks. Performance is evaluated using metrics such as task success rate, sample efficiency, and learning stability.

Task success rate is defined as the fraction of evaluation episodes in which the robot successfully completes the task. Formally, if N episodes are executed and $S_i \in \{0, 1\}$ denotes whether episode i was successful, the success rate is computed as

$$\text{Success Rate} = \frac{1}{N} \sum_{i=1}^N S_i.$$

In simulation, task success is determined using environment state variables. For example, in object manipulation tasks an episode is considered successful if the manipulated object reaches a predefined target region or configuration within the episode horizon.

We compare the proposed trajectory-guided reinforcement learning approach against several baselines. As a simple imitation-based baseline, we include behavioural cloning (BC), where the policy is trained in a supervised manner to directly predict the actions of an expert given the same observations. Formally, BC learns a policy $\pi_\theta(a|o)$ by minimizing the discrepancy between the predicted actions and the expert actions contained in a dataset of demonstrations.

In addition, we compare against reinforcement learning with sparse rewards and methods that incorporate additional guidance during training. Each trained policy is evaluated over $N = 100$ episodes with fixed random seeds.

In particular, we compare against the HIL-SERL approach, which incorporates human corrections during policy rollouts to guide exploration and improve learning efficiency.

A potential challenge when comparing with HIL-SERL is that its performance depends on the amount of human interaction and the frequency of operator corrections during training. To ensure a fair comparison, we plan to control these factors by fixing the number of demonstrations and limiting the number of human intervention steps during training.

In particular, we will follow a protocol similar to the one described in [8], collecting a small dataset of 20–50 demonstrations and allowing a limited number of operator corrections during policy rollouts. This setup ensures that both methods operate under comparable data and supervision constraints.

In contrast to HIL-SERL, our method replaces human feedback with trajectories generated by a pretrained robot policy, which serve as an automated source of task guidance during reinforcement learning.

4 Anticipated Results

We expect that incorporating trajectory-based guidance into the reward function will improve the efficiency of reinforcement learning when transferring skills between robot embodiments.

In particular, we hypothesize that policies trained with trajectory guidance will achieve higher task success rates and learn more stable behaviors compared to reinforcement learning with sparse rewards alone. By providing additional information about how the task can be completed, trajectory-based rewards may help the policy explore more effectively and avoid uninformative regions of the state space.

Furthermore, we expect that the proposed approach will achieve performance comparable to methods that rely on human feedback, such as HIL-SERL, while eliminating the need for manual intervention during training. If successful, this would demonstrate that trajectories generated by a source robot policy can serve as an effective automated substitute for human corrections.

Finally, we anticipate that the proposed method will improve sample efficiency, allowing policies for the target robot to be trained using fewer interactions with the environment. Such results would support the idea that embodiment-independent task information contained in end-effector trajectories can be reused to accelerate learning across different robotic platforms.

If the hypothesis is confirmed, the results may suggest that trajectory-based guidance is a promising direction for scalable cross-embodiment policy transfer.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning Transferable Visual Models From Natural Language Supervision,” Feb. 2021, arXiv:2103.00020 [cs]. [Online]. Available: <http://arxiv.org/abs/2103.00020>

- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” Jul. 2023, arXiv:2307.15818 [cs]. [Online]. Available: <http://arxiv.org/abs/2307.15818>
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An Open-Source Vision-Language-Action Model,” Sep. 2024, arXiv:2406.09246 [cs]. [Online]. Available: <http://arxiv.org/abs/2406.09246>
- [5] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “Smolvla: A vision-language-action model for affordable and efficient robotics,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.01844>
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” Aug. 2017, arXiv:1707.06347 [cs]. [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [7] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, “Soft Actor-Critic Algorithms and Applications,” Jan. 2019, arXiv:1812.05905 [cs]. [Online]. Available: <http://arxiv.org/abs/1812.05905>
- [8] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” Mar. 2025, arXiv:2410.21845 [cs]. [Online]. Available: <http://arxiv.org/abs/2410.21845>
- [9] Open X Collaboration, “Open X-Embodiment: Robotic Learning Datasets and RT-X Models,” May 2025, arXiv:2310.08864 [cs]. [Online]. Available: <http://arxiv.org/abs/2310.08864>
- [10] W. Liu, H. Zhao, C. Li, Y. Deng, J. Biswas, S. Pouya, and Y. Chang, “COMPASS: Cross-embodiment Mobility Policy via Residual RL and Skill Synthesis,” Oct. 2025, arXiv:2502.16372 [cs]. [Online]. Available: <http://arxiv.org/abs/2502.16372>
- [11] Y. Wang, X. Li, W. Wang, J. Zhang, Y. Li, Y. Chen, X. Wang, and Z. Zhang, “Unified Vision-Language-Action Model,” Jun. 2025, arXiv:2506.19850 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.19850>
- [12] J. Ho and S. Ermon, “Generative Adversarial Imitation Learning,” Jun. 2016, arXiv:1606.03476 [cs]. [Online]. Available: <http://arxiv.org/abs/1606.03476>
- [13] X. B. Peng, P. Abbeel, S. Levine, and M. v. d. Panne, “DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills,” *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1–14, Aug. 2018, arXiv:1804.02717 [cs]. [Online]. Available: <http://arxiv.org/abs/1804.02717>

Word count: 2825